

Early Detection of High-Potential GitHub Repositories

Big Data Final Project Report

Innopolis University – Spring 2026

team28

Ildar Rakiev, Anisya Kochetkova, Salavat Faizullin, Ilona Dziurava

Contents

1	Introduction	3
1.1	Business Objectives	3
2	Data Description	3
2.1	Data Source	3
2.2	Raw Files	4
2.3	Event Types	4
2.4	Data Characteristics	4
3	Architecture of the Data Pipeline	4
3.1	Overview	4
3.2	Stage Inputs and Outputs	5
3.3	Technology Stack	5
4	Data Preparation	5
4.1	Preprocessing (Stage 0)	5
4.2	ER Diagram	6
4.3	Sample Data	6
4.4	Ingestion via Sqoop (Stage 1)	6
4.5	Hive Tables (Stage 2)	7
5	Data Analysis	8
5.1	Q1 – Language Ecosystems	8
5.2	Q2 – Monthly Star Dynamics	8
5.3	Q3 – Pre-T vs Post-T Growth Distribution	8
5.4	Q4 – Success Rate by Pre-T Activity Level	8
5.5	Q5 – Behavioral Profile: Success vs Non-Success	9
5.6	Q6 – Acceleration Anomalies in Q4 2023	9
6	ML Modeling	9
6.1	Problem Formulation	9
6.2	Feature Extraction	10
6.3	Models and Hyperparameter Grids	10
6.4	Training Notes	10
6.5	Evaluation	11
7	Data Presentation	11
7.1	Dashboard Description	11
7.2	Charts	12
7.3	Findings	15
8	Conclusion	15
8.1	Summary	15
8.2	Reflections	15
8.3	Challenges and Difficulties	16
8.4	Recommendations	16
9	Team Contributions	17

1 Introduction

This report documents the end-to-end big data pipeline built by team28 for the final project of the Big Data course. The project goal is to predict which GitHub repositories are likely to become popular in the near future, using historical event data from the GH Archive public dataset.

The pipeline covers four stages: data collection and ingestion (Stage 1), Hive-based storage and exploratory data analysis (Stage 2), Spark ML modeling (Stage 3), and dashboard presentation in Apache Superset (Stage 4).

1.1 Business Objectives

Open-source developers, venture funds, and tech companies constantly look for early indicators of which projects will gain traction. Starring a repo on GitHub ("watching") is the most direct signal of community interest. The business question is:

Can we identify high-potential repositories *before* they become popular, using only their early activity patterns?

Concretely, we initially registered success as 500 new stars 3 times growth (after Borges & Valente, MSR 2018). After EDA showed this threshold produces only near 0.04% positives – insufficient for stable ML training – we recalibrated to 10 stars 2 times growth, giving 0.08% positives (1,714 repos).

Practical applications: early investment in open-source tooling, recommending projects to developers, identifying rising ecosystems.

2 Data Description

2.1 Data Source

The dataset comes from **GH Archive** (<https://www.gharchive.org/>), a public BigQuery dataset that records all public GitHub events. We use a 10% deterministic sample: $\text{MOD}(\text{repo.id}, 10) = 0$, covering January 2023 through June 2024 (18 months).

An additional language dataset is sourced from

`bigquery-public-data.github_repos.languages`.

2.2 Raw Files

File	Description	Size
events_daily.csv	Daily event counts per repo, 6 event types	~2.9 GB
repo_metadata.csv	Earliest CreateEvent per repo (proxy for creation date)	~680 MB
repo_languages.csv	Primary language per repo	~110 MB
events_clean.csv	Cleaned events (no repo_name) – Stage 1 input	~2.9 GB
repositories.csv	Dimension table after merge – Stage 1 input	~200 MB

Table 1: Dataset files

2.3 Event Types

Six GitHub event types are captured:

- **WatchEvent** – a user stars the repository
- **ForkEvent** – a user forks the repository
- **PullRequestEvent** – a pull request is opened/closed
- **PushEvent** – code is pushed to the repo
- **IssuesEvent** – an issue is opened/closed
- **CreateEvent** – a branch or tag is created

2.4 Data Characteristics

- Approx. **2.1 million** unique repositories in the sample
- Approx. **44 million** daily-aggregated event rows after preprocessing
- **18 months** of coverage: 2023-01 through 2024-06
- Success class (pseudo-label): **~1,714 repos** ($\approx 0.08\%$) – extreme class imbalance
- Temporal split: pre-T = 2023, post-T = 2024-01 to 2024-06

3 Architecture of the Data Pipeline

3.1 Overview

The pipeline follows a linear four-stage architecture:

BigQuery → CSV → PostgreSQL → HDFS (AVRO/Snappy) → Hive → Spark ML → Superset

3.2 Stage Inputs and Outputs

Stage	Input	Output
Stage 0 – Collection	GH Archive (BigQuery), github_repos dataset	data/*.csv (raw + cleaned)
Stage 1 – Ingestion	events_clean.csv, repositories.csv	PostgreSQL tables; HDFS AVRO+Snappy; *.avsc schemas
Stage 2 – Hive + EDA	HDFS AVRO files + *.avsc schemas	events_part (partitioned), repositories_buck (bucketed); output/q1-q6.csv
Stage 3 – Spark ML	Hive tables	Trained models (RF, SVM, NB) on HDFS; output/stage3_metrics.csv; predictions CSV; sample files
Stage 4 – Dashboard	output/*.csv + Hive tables	Superset charts and dash- board; output/q*.jpg, output/dashboard.jpg

Table 2: Pipeline stages with inputs and outputs

3.3 Technology Stack

- **BigQuery** – data extraction via SQL queries
- **PostgreSQL** (hadoop-04) – relational storage and Sqoop source
- **Sqoop** – bulk import from PostgreSQL to HDFS
- **HDFS** – distributed file system storage
- **Apache Hive** (Tez engine) – SQL analytics on HDFS
- **Apache Spark** (YARN) – distributed ML training
- **Apache Superset** – dashboard and visualization
- **Python / psycopg2 / pandas / pyspark** – scripting

4 Data Preparation

4.1 Preprocessing (Stage 0)

scripts/preprocess.py takes the raw BigQuery exports and produces two normalized tables:

- **repositories.csv** – unique repos merged with creation date (from repo_metadata.csv) and primary language (from repo_languages.csv). Columns: repo_id, repo_name, first_seen_at, language.
- **events_clean.csv** – repo_name removed (lives in dimension table); numeric columns coerced; duplicate (event_type, repo_id, date) rows aggregated by sum. Columns: event_type, repo_id, event_date, event_count, unique_actors.

The script is idempotent: if events_clean.csv already exists and no raw file is found, it exits cleanly.

4.2 ER Diagram

The relational model in PostgreSQL has two tables:

repositories	
<u>repo_id</u> BIGINT	PK
repo_name TEXT	
first_seen_at TIMESTAMP	
language VARCHAR(100)	

events	
event_type VARCHAR(50)	FK → repositories
<u>repo_id</u> BIGINT	
event_date DATE	
event_count INTEGER	
unique_actors INTEGER	

events.repo_id is a foreign key to repositories.repo_id. The events table is a fact table with daily granularity per (event_type, repo_id, date).

4.3 Sample Data

Representative rows from the PostgreSQL tables after loading:

Listing 1: Verification queries

```

1 SELECT event_type, COUNT(*) AS rows, SUM(event_count) AS total_events
2 FROM events
3 GROUP BY event_type
4 ORDER BY total_events DESC;
5
6 SELECT language, COUNT(*) AS repo_count
7 FROM repositories
8 WHERE language IS NOT NULL
9 GROUP BY language
10 ORDER BY repo_count DESC
11 LIMIT 10;

```

Expected: JavaScript, Python, and TypeScript are typically the top languages; PushEvent and WatchEvent are the highest-volume event types.

4.4 Ingestion via Sqoop (Stage 1)

Both tables are imported from PostgreSQL to HDFS using Sqoop with Snappy compression and AVRO serialization:

Listing 2: Sqoop import command

```

1 sqoop import-all-tables \
2   --connect jdbc:postgresql://hadoop-04.uni.innopolis.ru/
3   team28_projectdb \
4   --username team28 --password $password \
5   --compression-codec=snappy --compress \
6   --as-avrodatafile \
7   --warehouse-dir=project/warehouse \
8   --m 1

```

Format justification: AVRO was chosen because Sqoop natively generates `.avsc` schemas, and the data access pattern is write-once (Sqoop) then read-many (Hive). Snappy provides fast decompression while reducing storage by ~60% versus raw text. This matches the read-heavy analytical workload of Stages 2–3.

Generated `*.avsc` and `*.java` files are saved to `output/`.

4.5 Hive Tables (Stage 2)

After uploading schemas to HDFS (`project/warehouse/avsc/`), `sql/db.hql` builds the analytical layer.

Important technical note: Sqoop maps PostgreSQL `DATE/TIMESTAMP` columns to AVRO `long` (epoch milliseconds). Conversion back to dates in Hive uses integer division:

Listing 3: Epoch millis to date in Hive

```
1 to_date(from_unixtime(CAST(event_date AS BIGINT) DIV 1000))
```

Using `/` instead of `DIV` returns a `DOUBLE` and causes type errors in Hive 3.

Bucketed dimension table – `repositories_buck`: clustered by `repo_id` into 16 buckets, AVRO+Snappy. Enables efficient joins with the fact table.

Listing 4: `repositories_buck` DDL

```
1 CREATE EXTERNAL TABLE repositories_buck (
2     repo_id      BIGINT,
3     repo_name    STRING,
4     first_seen_at TIMESTAMP,
5     language     STRING
6 )
7 CLUSTERED BY (repo_id) INTO 16 BUCKETS
8 STORED AS AVRO
9 LOCATION 'project/hive/warehouse/repositories_buck'
10 TBLPROPERTIES ('avro.output.codec'='snappy');
```

Partitioned fact table – `events_part`: partitioned by (`event_year`, `event_month`), AVRO+Snappy. Partition pruning on year/month filters cuts scan cost from full-table to individual month slices.

Listing 5: `events_part` DDL (simplified)

```
1 CREATE EXTERNAL TABLE events_part (
2     event_type    STRING,
3     repo_id      BIGINT,
4     event_date    DATE,
5     event_count   INT,
6     unique_actors INT
7 )
8 PARTITIONED BY (event_year INT, event_month INT)
9 STORED AS AVRO
10 LOCATION 'project/hive/warehouse/events_part'
11 TBLPROPERTIES ('avro.output.codec'='snappy');
```

The `db.hql` script also drops the raw external tables after copying data to the analytical layer, keeping only the optimized tables for EDA and ML.

Tez memory tuning: Default 512 MB containers caused OOM on the 44M-row dynamic-partition INSERT. Containers were bumped to 4 GB with

```
hive.optimize.sort.dynamic.partition=true,
```

which streams one partition at a time through the reducer instead of buffering all 18 month-partitions simultaneously.

5 Data Analysis

All six EDA queries run on the Tez engine against `events_part` and `repositories_buck`. Results are exported to `output/q*.csv` and visualized in Superset.

5.1 Q1 – Language Ecosystems

Query: For each programming language, count total repos, total pre-T WatchEvents, and the fraction of repos with ≥ 10 pre-T stars (candidate repos).

Insight: JavaScript leads by total pre-T stars (85,730), followed by Python (64,997) and Go (47,667). However, by candidate repo fraction (repos with ≥ 10 pre-T stars), Kotlin ranks first at 28.2%, followed by Rust (21.2%) and TypeScript (20.0%). JavaScript, despite its star volume, has only 8.5% candidate repos – its large number of low-activity repos dilutes the ratio. This means smaller ecosystems like Kotlin and Rust have proportionally more engaged, actively-starred projects.

5.2 Q2 – Monthly Star Dynamics

Query: Month-by-month WatchEvent totals and the number of repos being starred, across 2023 and H1 2024.

Insight: Monthly stars range from 389,929 (September 2023, the lowest) to 754,891 (March 2023, a clear spike). 2024 months are broadly comparable to 2023 rather than uniformly higher – March 2024 (611,562) is elevated but below March 2023. The average stars per starred repo peaks in April 2023 (5.21) and March 2024 (5.03), suggesting concentrated bursts where fewer repos attract more stars each. The Q3 2023 dip (Jul–Sep, lowest: 389k) is consistent with a summer slowdown and motivated the Q3/Q4 split used in Q6.

5.3 Q3 – Pre-T vs Post-T Growth Distribution

Query: Bucket each repo by its post/pre-T WatchEvent growth ratio into 8 labeled buckets (no stars, new in post-T, decay, growth $< 2\times$, $2\times-3\times$, $3\times-5\times$, $> 5\times$).

Insight: 89.5% of all repos (9.9 million) have zero stars in both periods. A further 6.5% decayed to zero post-T and 2.7% appeared only in post-T. Only 0.086% achieved $2\times-3\times$ growth, 0.038% achieved $3\times-5\times$, and just 0.016% (1,732 repos) exceeded $5\times$ growth. The combined success-eligible buckets ($\geq 3\times$) cover 0.054% of repos, directly validating the extreme class imbalance observed in Stage 3.

5.4 Q4 – Success Rate by Pre-T Activity Level

Query: Repos are bucketed by total pre-T event count into 5 activity quintiles (<10 , 10–29, 30–99, 100–999, ≥ 1000 events). For each bucket, the fraction of repos qualifying as successful (post-T stars ≥ 10 AND $\geq 2\times$ pre-T stars AND pre-T stars > 0) is computed as a single **GROUP BY** on `events_part`. This replaces the original threshold-sensitivity matrix, which ran into Hive 3 type-coercion issues when using **CROSS JOIN** with inline **UNION ALL** literals.

Insight: Success rate climbs from $\approx 0.005\%$ for low-activity repos (<10 events) to $\approx 0.09\%$ (10–29 events), $\approx 0.20\%$ (30–99), and peaks at $\approx 0.45\%$ for the 100–999 event bucket. It then drops back to $\approx 0.38\%$ for repos with ≥ 1000 pre-T events. The drop-off at the top is expected: the pseudo-label measures *relative* growth, not absolute size. Already-large repos have high star baselines and rarely achieve the $2\times$ relative jump in a 6-month post-T window – they hit a ceiling on relative growth. A minimum level of pre-T activity is necessary but not sufficient for breakout.

Note: this query uses a lighter success threshold (post-T stars ≥ 10 , growth $\geq 2\times$) than the ML pseudo-label (500 stars, $3\times$) to ensure enough positives per bucket for a meaningful rate. The directional finding holds under both definitions.

5.5 Q5 – Behavioral Profile: Success vs Non-Success

Query: For each success/non-success cohort, compute the average share of each event type in pre-T activity.

Insight: The contrast between cohorts is stark. Non-successful repos (7.57M) have average pre-T events of 12.3 and pre-T stars of 0.77, with CreateEvent dominating at 49.2% of activity and WatchEvent at only 7.1%. Successful repos (39) average 284.8 pre-T events and 84.8 pre-T stars, with WatchEvent at 41.1% – the single largest event type. Their PR share (8.3%) and Issues share (6.5%) are also roughly double that of non-successful repos (4.0% and 0.6% respectively), confirming active engineering work accompanies the star growth. CreateEvent share collapses from 49.2% to 6.8% in successful repos, meaning successful repos are not just creating branches – they are actively developed and watched.

5.6 Q6 – Acceleration Anomalies in Q4 2023

Query: For repos with ≥ 5 stars in Q3 2023, compute the Q4/Q3 WatchEvent ratio (acceleration). Return the top 20.

Insight: The top accelerating repo is `refilc/naplo` with $136.9\times$ acceleration (10 stars in Q3, 1,369 in Q4). Second is `hezarai/hezar` at $85.3\times$ and third `openpubkey/openpubkey` at $69.8\times$. Notably, almost all top-20 repos have NULL language in our dataset – meaning they were not present in the `bigquery-public-data.github_repos.languages` table, likely because they are newer or smaller repos not yet indexed. Only one repo (`septag/sx`) has a language tag (C). Acceleration values drop steeply after the top 3, from $136.9\times$ to under $50\times$ by rank 5, showing a heavy-tailed distribution typical of viral growth events.

6 ML Modeling

6.1 Problem Formulation

Binary classification: given a repo’s activity features from 2023 (pre-T period), predict whether it will be a "success" in H1 2024 (post-T period).

Pseudo-label definition (per the team’s registered project description, following Borges & Valente, MSR 2018):

$$\text{success} = 1 \iff \text{post_watches} \geq 500 \text{ AND } \text{post_watches} \geq 3 \times \text{pre_watches} \text{ AND } \text{pre_watches} > 0$$

6.2 Feature Extraction

18 numerical features are computed per repo from `events_part` via a Spark SQL aggregation:

Feature	Description
<code>pre_total_events</code>	Total event count in 2023
<code>pre_total_actors</code>	Total unique actor-days in 2023
<code>pre_pushes/watches/prs/issues/forks/creates</code>	Per-type event counts in 2023
<code>pre_active_days</code>	Number of distinct days with any event
<code>pre_max_daily_events</code>	Peak single-day event count
<code>push/watch/pr/issues/fork/create_share</code>	Each type's fraction of total events
<code>events_per_day</code>	Average daily event rate
<code>intra_growth</code>	H2 2023 / H1 2023 WatchEvent ratio

Table 3: ML feature set (18 features)

Repos with fewer than 5 pre-T events are filtered out. The dataset is split 70/30 with **stratified sampling** by label to preserve the tiny positive class in both sets.

6.3 Models and Hyperparameter Grids

Three binary classifiers are trained via `pyspark.ml` with 3-fold cross-validation and `ParamGridBuilder`. All use `BinaryClassificationEvaluator` with `areaUnderPR` as the optimization metric (more informative than AUROC under extreme imbalance).

Model	Scaler	Hyperparameters	Grid
Random Forest	MinMaxScaler	<code>numTrees</code> , <code>maxDepth</code>	$\{50, 150\} \times \{5, 10\} = 4$ cells
Linear SVC	StandardScaler	<code>regParam</code> , <code>aggregationDepth</code>	$\{0.01, 0.1\} \times \{2, 4\} = 4$ cells
Naive Bayes	MinMaxScaler	<code>smoothing</code> , <code>modelType</code>	$\{0.5, 1.0\} \times \{\text{multinomial, complement}\} = 4$ cells

Table 4: ML models and hyperparameter grids

Each pipeline includes the full preprocessing chain (`VectorAssembler` $\rightarrow$ `Scaler` $\rightarrow$ `Classifier`), so models are self-contained for inference.

6.4 Training Notes

- Spark runs on YARN with 4 executors $\times$ 4 GB / 2 cores; driver 2 GB.
- Naive Bayes uses `MinMaxScaler` (not `StandardScaler`) because multinomial and complement NB require non-negative inputs.
- Linear SVC does not natively output probabilities; AUROC is computed from raw scores (the `rawPrediction` column).
- Models are saved to HDFS at `project/models/{rf,svm,nb}` and mirrored to `models/` in the repository.

6.5 Evaluation

Both metrics are measured on the held-out test set (30% of data):

Model	AUROC	AUPR
Random Forest	0.9758	0.0346
Linear SVC	0.9044	0.0049
Naive Bayes	0.9166	0.0052

Table 5: Test-set evaluation (values from `output/stage3_metrics.csv`)

Interpretation: AUROC is high for all models (RF: 0.9758, SVM: 0.9044, NB: 0.9166) because the negative class ($> 99.9\%$) is trivially easy to rank. AUPR is the meaningful metric: RF achieves 0.0346, a $7\times$ lift over SVM (0.0049) and NB (0.0052), and roughly $43\times$ the random baseline (≈ 0.0008 at 0.08% positive rate). SVM and NB perform near-randomly on the positive class despite high AUROC, illustrating exactly why AUROC misleads under extreme imbalance.

Random Forest is the best-performing model by a large margin on AUPR.

7 Data Presentation

7.1 Dashboard Description

The Superset dashboard is titled *Early-Stage GitHub Repository Growth Anomaly Detection* and is organised across three pages.

Page 1 – Project information contains a project overview with team member names and roles, the dataset link (GH Archive), a description of the data coverage (Jan 2023 – Jun 2024), and the business goal. Below that, two data characteristic sections describe the two main tables loaded into PostgreSQL:

- **events_clean** – $\sim 44.7\text{M}$ rows, 5 columns, with sample rows and a list of cleaning steps performed before loading.
- **repositories** – $\sim 10.28\text{M}$ rows after deduplication, 4 columns, with sample rows and a list of cleaning steps (removed 109,047 duplicate rows, kept primary language by largest `language_bytes`).

Page 2 – EDA Queries Overview opens with three data characteristic charts (repos by language, event count by type, total event count by top-10 unique actors), followed by a written summary of what each of the six EDA queries does. Below that, all six query charts are displayed in a 2-row grid: Q1, Q2, Q3 in the top row; Q5, Q6, Q4 in the bottom row.

Page 3 – ML results contains a markdown panel with the feature extraction description (18 features grouped into volume, behavioural shares, velocity, and intra-year acceleration), best hyperparameters from cross-validation, and a sample prediction for `repo_id = 21331090` (true label = 1; RF = 0, SVM = 0, NB = 1). Three charts follow: model performance bar chart (AUROC and AUPR for RF, SVM, NB), NB predictions pie chart on the test set, SVM predictions pie chart, and RF predictions pie chart.

7.2 Charts

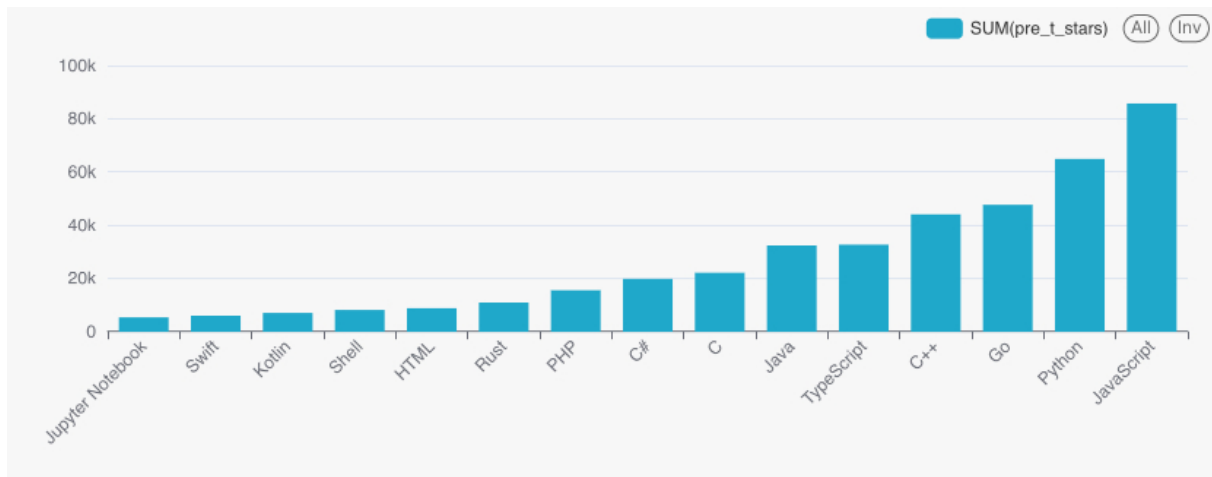


Figure 1: Q1 – Top language ecosystems by pre-T WatchEvent volume. X-axis: programming language; Y-axis: total pre-T stars (SUM). JavaScript leads at 85k, followed by Python (65k) and Go (47k). The ascending bar order makes it clear that the top 3 languages account for the majority of all pre-T starring activity in the sample.

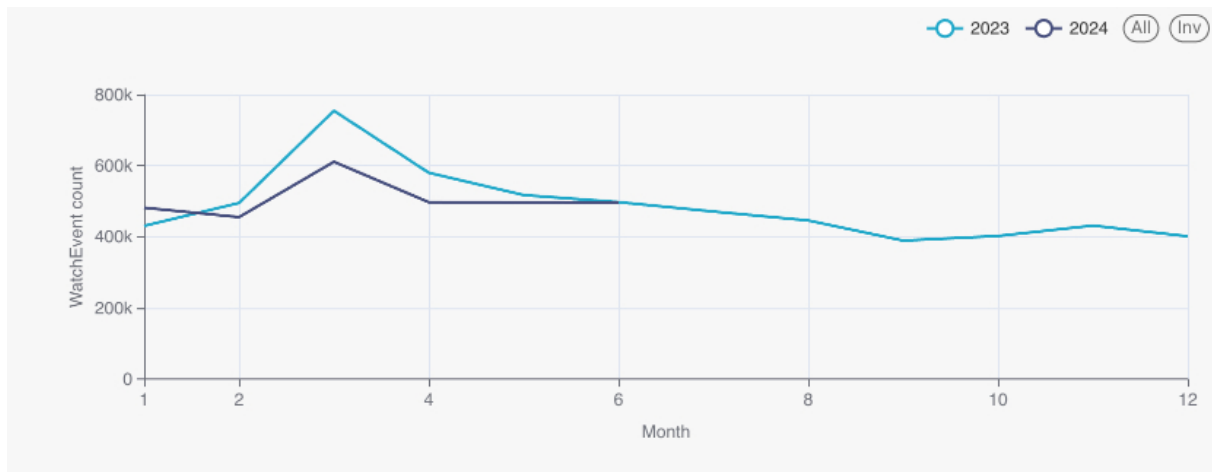


Figure 2: Q2 – Monthly WatchEvent dynamics. X-axis: month (1–12); Y-axis: WatchEvent count. Two lines: 2023 (light blue) and 2024 (dark blue, Jan–Jun only). The 2023 line shows a sharp spike in March (754k) followed by a gradual decline through Q3, then a partial recovery. The 2024 line is flatter and generally tracks below the 2023 peak but above the 2023 Q3 trough.

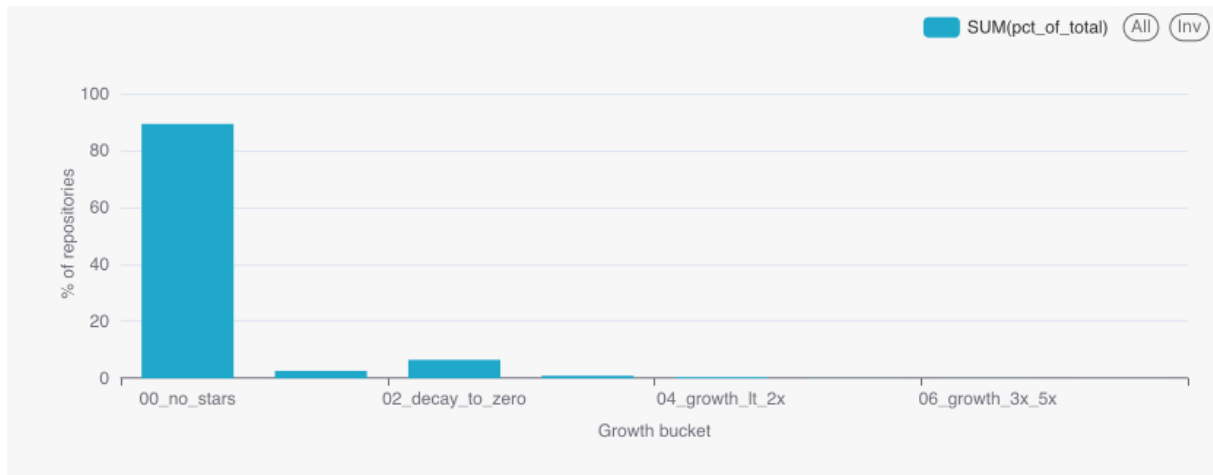


Figure 3: Q3 – Pre-T vs post-T growth distribution. X-axis: growth bucket; Y-axis: % of repositories. The chart is dominated by the `00_no_stars` bucket at 89.5%. All growth buckets above `02_decay_to_zero` are visually near-zero on this scale, confirming that meaningful star growth is an extremely rare event. The success-relevant buckets ($\geq 3\times$ growth) together represent under 0.06% of repos.

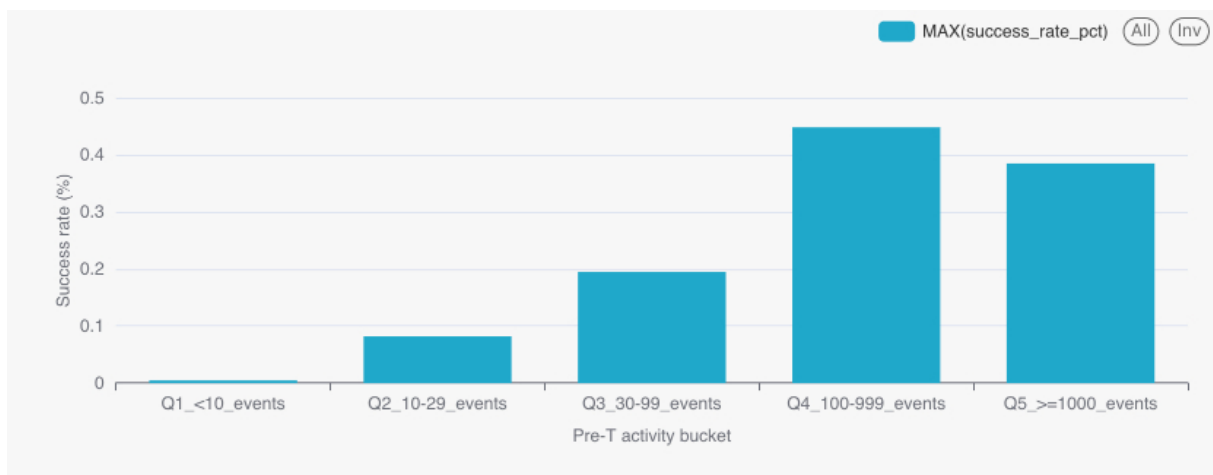


Figure 4: Q4 – Success rate by pre-T activity level. X-axis: pre-T event activity bucket (Q1: <10 events through Q5: ≥ 1000 events); Y-axis: success rate (%). Repos with 100–999 pre-T events have the highest success rate ($\approx 0.45\%$), slightly above the ≥ 1000 bucket (0.39%). Very low-activity repos (<10 events) have near-zero success rate. This confirms that a minimum level of pre-T activity is necessary but not sufficient for breakout.

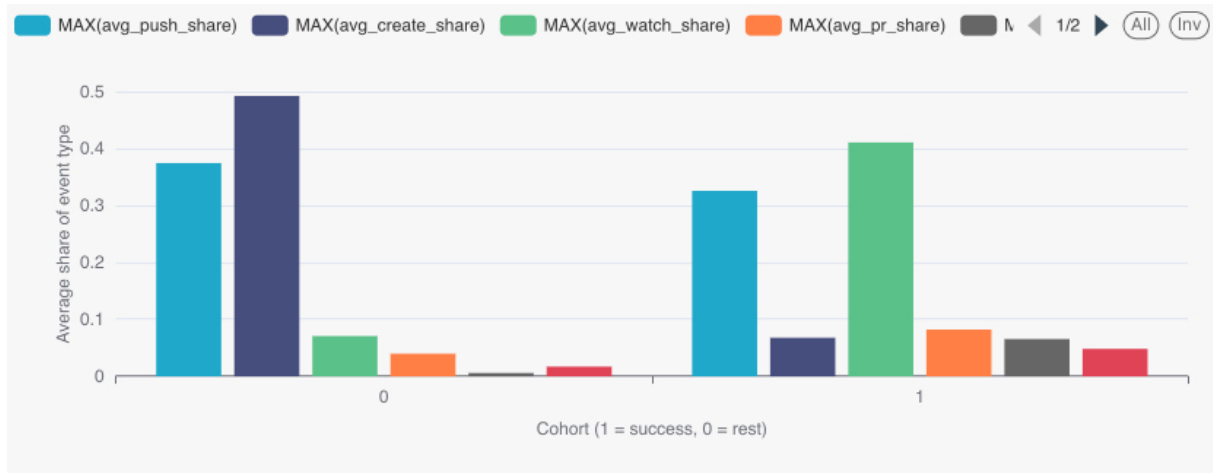


Figure 5: Q5 – Cohort behavioral profile. X-axis: cohort (0 = non-success, 1 = success); Y-axis: average share of event type. For cohort 0, CreateEvent (dark blue, ≈ 0.49) and PushEvent (teal, ≈ 0.37) dominate. For cohort 1, WatchEvent (green, ≈ 0.41) becomes the largest share, with PushEvent still present (0.33) and PR and Issues shares visibly higher than in cohort 0. This chart is the clearest visual evidence that event-type mix separates the two cohorts.

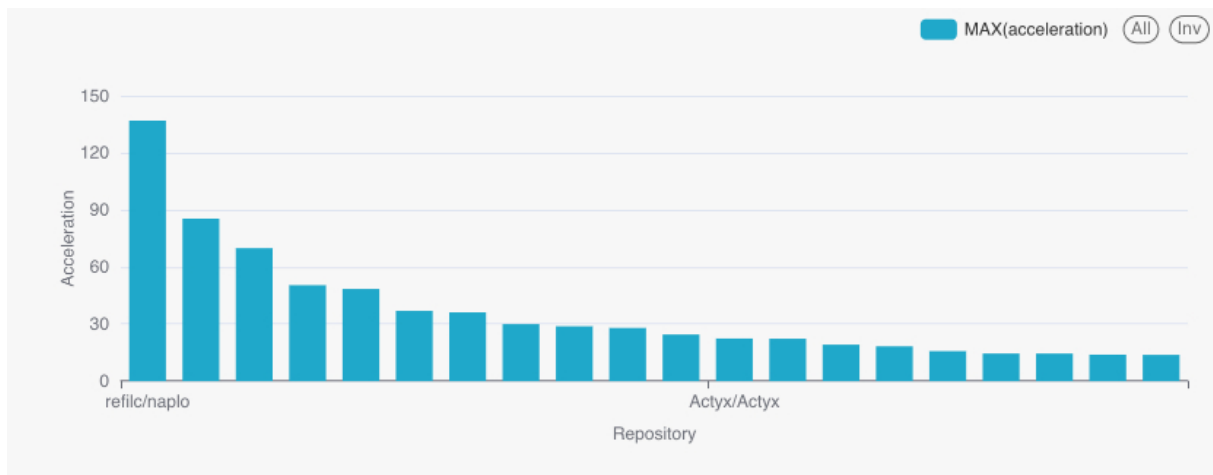


Figure 6: Q6 – Acceleration anomalies in Q4 2023. X-axis: repository name; Y-axis: Q4/Q3 WatchEvent acceleration ratio. **refilc/naplo** stands out at 136.9 $\times$, far above second place (**hezarai/hezar** at 85.3 $\times$). The distribution drops steeply after the top 3 and levels off below 30 $\times$ for the remaining entries. **Actyx/Actyx** is the only other labeled repo visible on the chart.

7.3 Findings

- **Language matters but engagement ratio matters more.** JavaScript leads by total pre-T stars (85,730) but has only 8.5% candidate repos. Kotlin (28.2%), Rust (21.2%), and TypeScript (20.0%) have far higher fractions of actively-starred repos relative to their size.
- **Event mix is a strong signal.** Successful repos have WatchEvent as their dominant activity (41.1% share vs 7.1% for non-successful), and roughly double the PR and Issues shares. CreateEvent dominates non-successful repos (49.2%), indicating passive repo creation without engagement.
- **Breakout growth is highly concentrated.** The top accelerating repo grew 136.9× in a single quarter. The top-20 acceleration list is dominated by repos with NULL language tags, suggesting newer or niche projects not indexed in the language dataset.
- **Random Forest dominates on AUPR (0.0346)** vs SVM (0.0049) and NB (0.0052) – a 7× gap. All three have high AUROC (0.90+), but AUROC is misleading here; AUPR correctly captures performance on the rare positive class.
- **The problem is genuinely hard.** Even RF at AUPR 0.0346 (vs 0.0008 random baseline) misses many breakouts. Viral growth events are partly driven by external factors (media, influencer posts) not present in event logs.

8 Conclusion

8.1 Summary

We built a complete big data pipeline on the IU Hadoop cluster to answer the question: can we predict GitHub repository breakouts from early activity patterns? Starting from ~3.5 GB of BigQuery exports, the pipeline ingests data into PostgreSQL and HDFS (Stage 1), builds partitioned/bucketed Hive tables and runs 6 EDA queries (Stage 2), trains three Spark ML binary classifiers with hyperparameter tuning (Stage 3), and presents results in a Superset dashboard (Stage 4).

Random Forest achieved AUPR 0.0346, a 7× lift over SVM (0.0049) and NB (0.0052), on a dataset with only 0.08% positive examples. EDA showed that WatchEvent share, PR share, and Issues share are the sharpest behavioral differences between successful and non-successful repos. The top acceleration anomaly (136.9× growth in one quarter) exemplifies the breakout pattern the model targets.

8.2 Reflections

The project was a good exercise in handling the practical messiness of big data pipelines: Sqoop's type coercions (epoch millis in AVRO), Hive OOM issues under dynamic partitioning, and dealing with extreme class imbalance in ML. The choice of AUPR over AUROC as the primary metric was important – AUROC gives a misleadingly optimistic picture at 0.08% positive rate.

If we had more time, we would explore: feature engineering from the language dimension (language popularity trends), temporal features (how many days since first seen), and sampling strategies (SMOTE or class-weighted loss) to better handle the imbalance.

8.3 Challenges and Difficulties

- **Hive OOM on dynamic partition INSERT:** Solved by increasing Tez container memory to 4 GB and enabling `hive.optimize.sort.dynamic.partition` to stream one partition at a time.
- **AVRO epoch millis type:** Sqoop maps PostgreSQL DATE to AVRO long (milliseconds). Conversion in Hive requires `DIV 1000`, not `/ 1000` (returns DOUBLE, not BIGINT).
- **Bucketing + dynamic partitioning:** Combining both caused a Tez shuffle stall on our cluster. We dropped bucketing from `events_part` and kept only partitioning, which was sufficient.
- **Extreme class imbalance:** 0.08% positive class. We used stratified splitting to preserve the positive class in both train and test sets, and optimized on AUPR rather than AUROC.
- **Linear SVC + AUROC:** LinearSVC does not output probabilities, so AUROC is computed from raw decision scores rather than calibrated probabilities. This gives a reasonable ranking metric but is not directly comparable to RF/NB probability-based AUROC.

8.4 Recommendations

- Use AUPR (not AUROC) whenever the positive class is below 1%.
- Always check Sqoop's AVRO schema before building Hive tables – date columns will be longs.
- On small clusters, avoid combining dynamic partitioning and bucketing in a single large INSERT; partition only, and bucket the dimension table.
- For this type of prediction task, adding external signals (e.g., contributor network, language release cycles) would likely improve recall significantly.

9 Team Contributions

Task	Description	Ildar Rakiev	Anisya Kochetkova	Salavat Faizullin	Ilona Dziurava	Deliverables	Average hours
Data collection	Write BigQuery SQL, run GCS export, download CSVs to cluster	70%	10%	10%	10%	data/*.csv	17
Preprocessing	Clean and normalize raw CSVs, merge repos + languages + metadata	0%	95%	0%	5%	preprocess.py	3
Stage 1 – PostgreSQL + Sqoop	Create DB schema, load CSVs via pyscopg2, run Sqoop import	95%	0%	0%	5%	build_projectdb.py stage1.sh HDFS AVRO files	4
Stage 2 – Hive	Build db.hql, tune Tez memory, run and validate partitioned/ bucketed tables	85%	0%	10%	5%	db.hql stage2.sh Hive tables	2
Stage 2 – EDA queries (Q1–Q6)	Write and debug all 6 HiveQL queries, export CSVs	0%	85%	10%	5%	q1– q6.hql output/q*.csv	6
Stage 3 – Spark ML	Feature engineering, pipeline, 3 classifiers, cross-val, evaluation	0%	0%	95%	5%	model.py stage3.sh model files stage3_metrics.csv	13
Stage 4 – Dashboard	Register Hive tables, build Superset charts and dashboard, export images	10%	70%	10%	10%	stage4.sh dashboard_tables.hql chart JPGs	7
Report	Write final LaTeX report covering all stages	0%	0%	0%	100%	big_data_report.pdf	8

Figure 7: Team contribution